

Receipt:

$$e_t = \mathcal{G}(z_t) = \begin{cases} \text{committed,} & z_t = \text{yes,} \\ \text{rejected,} & z_t = \text{no,} \\ \text{unknown,} & z_t = \text{uncertain.} \end{cases} \quad (10)$$

The receipt stores the action identity, software status, key evidence, and physical judgment. It is the only interface that can authorize trusted memory updates.

3.4 Remember with Receipt-Gated Bounded Memory

The memory mechanism contains verified memory, verified progress, an attempt log, and relation memory. Only $e_t = \text{committed}$ updates verified world facts and task progress. A rejected or unknown receipt leaves trusted state unchanged. The system records the action, evidence, receipt type, and attempt count in the attempt log. A rejected receipt means that the intended effect is confirmed absent. An unknown receipt means that the outcome remains unresolved. Relation memory stores object relations, skill relations, and task constraints when such relations are available. The system combines these relations with verified facts, verified progress, and typed diagnostics to form the bounded context:

$$\mathcal{B}_{t+1} = C(\mathcal{M}_{t+1}^{\text{ver}}, \mathcal{P}_{t+1}^{\text{ver}}, \mathcal{R}_{t+1}, \mathcal{D}(\mathcal{L}_{t+1})). \quad (11)$$

Here, $\mathcal{L}$ denotes the attempt log and $\mathcal{D}$ extracts diagnostic summaries. Diagnostics help the planner avoid ineffective strategies. They cannot be promoted to physical facts. We impose a budget B on $\mathcal{B}_t$. The bounded view preserves recent observations, committed progress, and selected receipt-linked evidence, while the complete audit information remains outside the planner context in the attempt log.

3.5 Recovery through Re-observation and Reconciliation

Recovery selects the next procedure from the admission result, effect receipt, attempt log, and bounded context. Every path follows the same rule. A physical retry requires a fresh observation. For an outdated plan step, the system rejects dispatch and regenerates the plan suffix while preserving the committed prefix. For a rejected receipt, the system may choose an alternative skill, replan, or perform a bounded safe retry. The attempt log prevents repeated use of a strategy that has already failed. For an unknown receipt, the system executes no new task action solely from the unresolved outcome. It first obtains fresh evidence from the adapter, such as a new camera frame, evaluator state, or operator effect label. After receiving a new observation o'_t , the system forms a fresh report ρ'_t and recomputes $z'_t = f_{\text{fuse}}(\rho'_t, o'_t, g_t)$.

A yes result produces a committed receipt and resumes the task. A no result produces a rejected receipt and starts safe recovery. If uncertainty remains, the system gathers more evidence. After N_{obs} attempts, it marks the outcome as unresolved and requests stronger perception or human confirmation. A non-idempotent skill is never repeated directly from an unknown state. This rule prevents duplicate grasping, pushing, and placing effects.

Table 2: Evaluation metrics and their definitions.

Metric	Definition
Task Success Rate (TSR) $\uparrow$	Upstream task-evaluator judgment reported without modification.
Verified Success Rate (VSR) $\uparrow$	Rate of task-successful episodes whose execution chain has no outdated plan-step dispatch, duplicate dispatch, or unresolved outcome.
TSR-VSR Gap $\downarrow$	Declared successes that the harness cannot support with its own evidence.
Unverified Memory Writes $\downarrow$	Outcomes admitted to trusted memory without a verified physical effect.
Duplicate Physical Actions $\downarrow$	Logical invocations that produce more than one physical dispatch.
Recovery Coverage $\uparrow$	Rate of eligible episodes that enter recovery.
Recovery Success $\uparrow$	Rate of recovery cases that are resolved.
Context and Execution Cost $\downarrow$	Actions, model calls, wall-clock time, max images per request, and peak request size.

3.6 Implementation Details

All embodiments use the same pipeline for planning, verification, receipt gating, memory maintenance, and recovery. Components tied to the physical platform remain embodiment-specific. These include the skill library, state constraints, effect measurements, and observation tools. In simulation [4, 24, 34], the verifier directly reads object, end-effector, and robot-base poses from the environment. For dexterous hands, the state also includes finger joints, contact states, and palm pose. For gripper-based manipulators, it includes gripper width, contact force, and end-effector pose. For mobile robots, it includes base pose, goal distance, and residual velocity. The model configuration follows the native setup of each benchmark. Clutter-Bench uses Qwen3-VL-32B-Instruct [37] with a temperature of 0.2, an 800-token output limit, and 512x512 rendered PNG frames with low image detail. We allow up to three provider retries and set the planning budget to 60 turns. LIBERO uses a frozen SmolVLA [33] checkpoint as its action policy rather than an API-based VLM planner. The navigation experiments retain their native macro-level stack. GPT-4o [16] serves as the language-action model, while Qwen3.5-27B [37] serves as the vision-action model with reasoning output disabled.

For real-world execution, vision and robot sensors jointly estimate physical effects. DINOv2 features [28] provide target point prompts, and SAM [18] extracts object masks. Depth observations are then back-projected to estimate rigid transformations between consecutive point clouds. Wider effect tolerances account for perception noise. Within each benchmark, all variants share the same planner or policy, seeds, initial states, and interaction budget. Retry limits, re-observation limits, memory budgets, effect thresholds, and embodiment constraints are selected on validation data and fixed within each embodiment class.

4 Experiments

Our experiments evaluate RoboGuardian through four questions. **Q1:** Does RoboGuardian preserve task capability under fault-free execution? **Q2:** Can it prevent unverified updates and duplicate actions under execution exceptions? **Q3:** Can receipt-gated memory preserve verified progress while bounding context and execution cost? **Q4:** Does the framework transfer across action granularities, task modalities, robot embodiments, and physical platforms?